

```

def n_queens(n):
    col = set()
    posDiag=set() # (r+c)
    negDiag=set() # (r-c)

    res=[]

    board = [["0"]*n for i in range(n) ]

    def backtrack(r):
        if r==n:
            copy = [" ".join(row) for row in board]
            res.append(copy)
            return

        for c in range(n):
            if c in col or (r+c) in posDiag or (r-c) in negDiag:
                continue

            col.add(c)
            posDiag.add(r+c)
            negDiag.add(r-c)
            board[r][c]="1"

            backtrack(r+1)

            col.remove(c)
            posDiag.remove(r+c)
            negDiag.remove(r-c)
            board[r][c]="0"

    backtrack(0)

    for sol in res:

```

```
for row in sol:
```

```
    print(row)
```

```
print()
```

```
if __name__=="__main__":
```

```
    n_queens(8)
```

```
1 def n_queens(n):
2     col = set()
3     posDiag=set() # (r+c)
4     negDiag=set() # (r-c)
5
6     res=[]
7
8     board = [["0"]*n for i in range(n) ]
9     def backtrack(r):
10         if r==n:
11             copy = [" ".join(row) for row in board]
12             res.append(copy)
13             return
14
15         for c in range(n):
16             if c in col or (r+c) in posDiag or (r-c) in negDiag:
17                 continue
18
19             col.add(c)
20             posDiag.add(r+c)
21             negDiag.add(r-c)
22             board[r][c]="1"
23
24             backtrack(r+1)
25
26             col.remove(c)
```

```
10000000
00001000
00000001
00000100
00100000
00000010
01000000
00010000
10000000
00000100
00000001
00100000
00000010
00010000
01000000
00001000
10000000
00000010
00010000
00000100
00000001
01000000
00001000
```